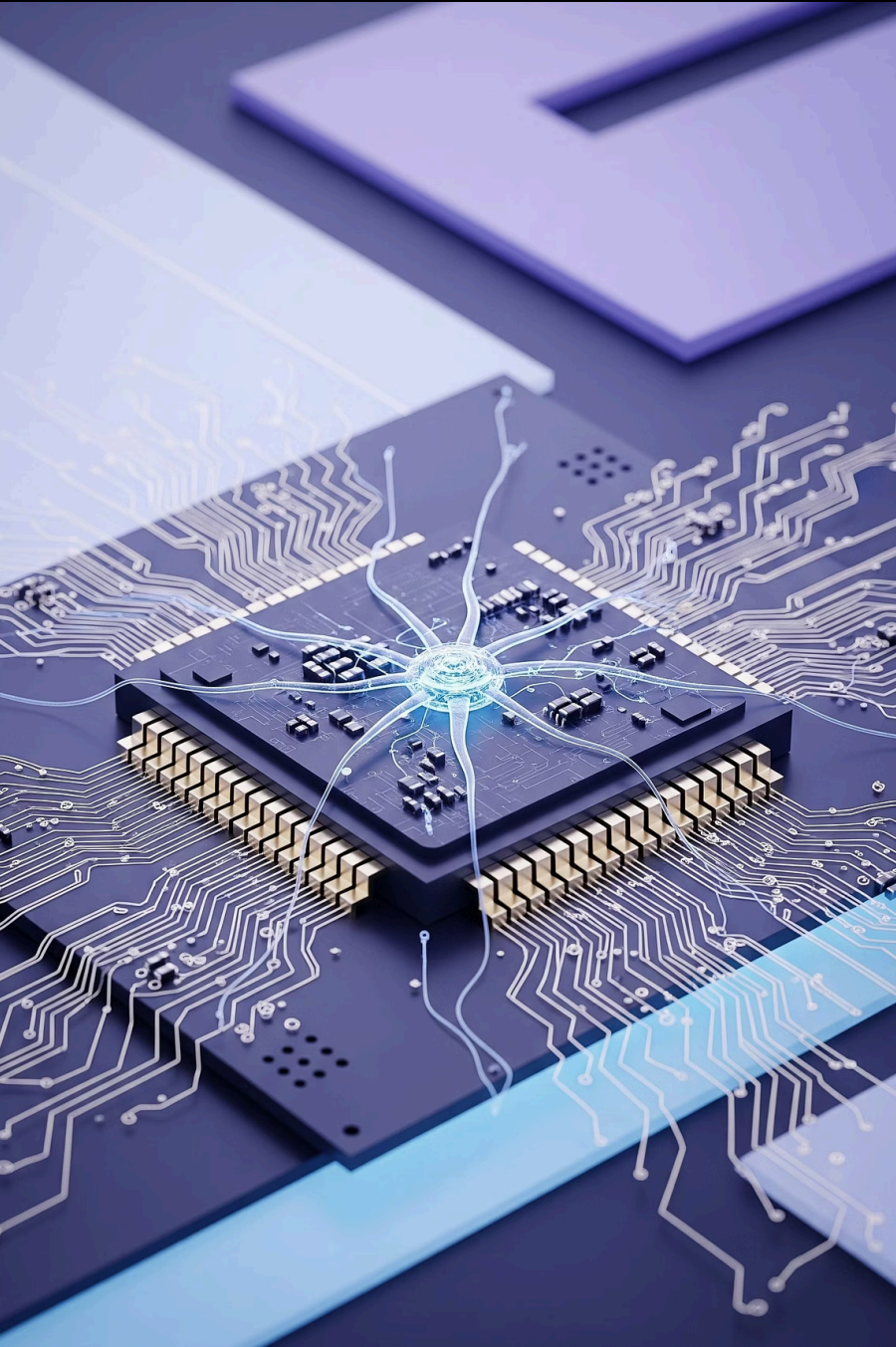




Quantization for Edge Efficiency

Smaller, faster models for resource-constrained devices

Optimize ML model deployment on resource-constrained hardware through quantization techniques that maintain accuracy while dramatically reducing computational requirements.

Why Quantize?



Latency ↓

INT8 operations execute significantly faster than FP32 on many edge processors



Memory ↓

Weights and activations shrink dramatically (4× reduction from FP32→INT8)



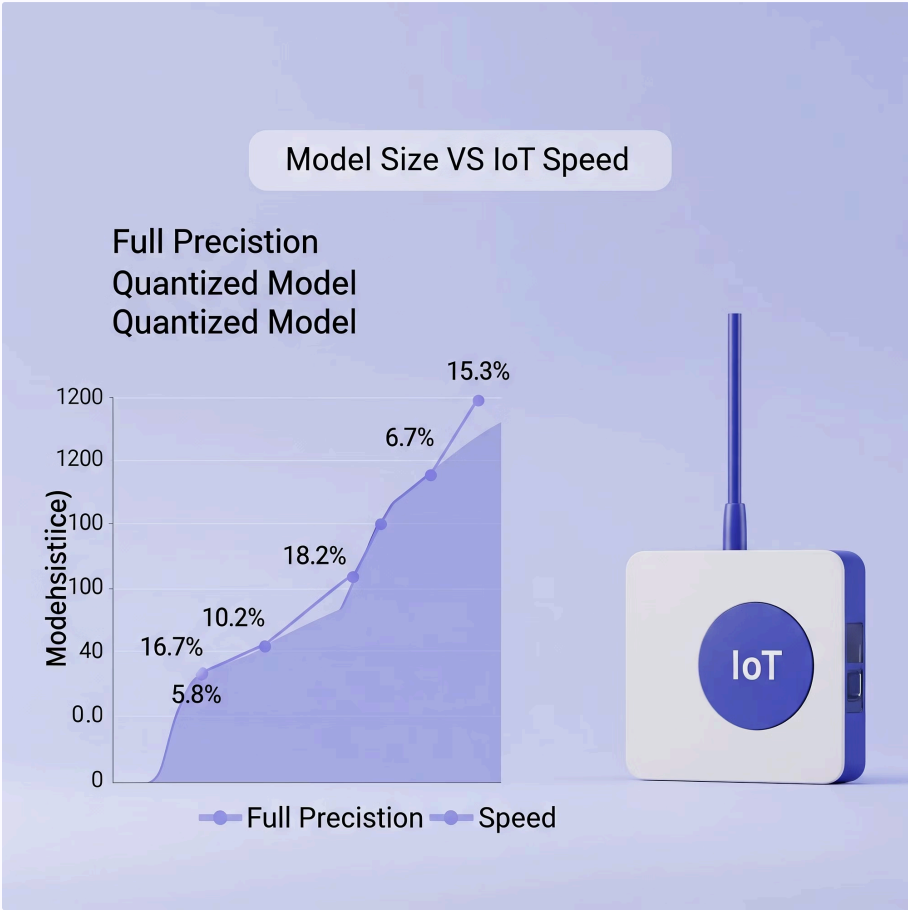
Power ↓

Fewer bits = less energy consumed per operation



Deployability ↑

Fit sophisticated models on microcontroller and SoC memory budgets



Key Quantization Concepts



Bit-width Reduction

FP32 → FP16/BF16 → INT8/INT4

Progressively trade precision for efficiency



Quantization Parameters

Maps real values to integers using **scale** and **zero-point**

$q = \text{round}(x/\text{scale}) + \text{zero_point}$

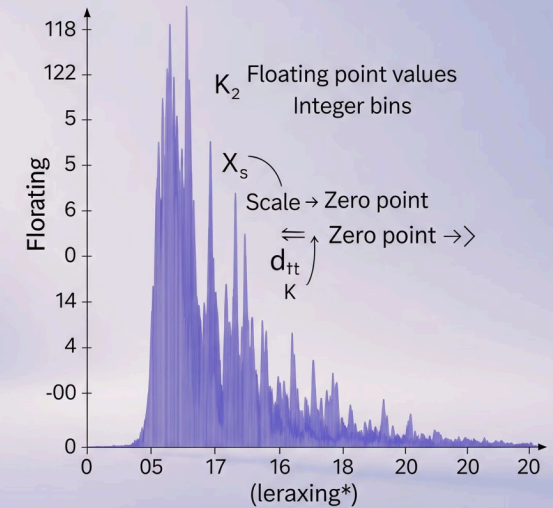


Granularity Choices

Per-tensor vs per-channel quantization

Per-channel typically more accurate for weights

Quantization mapping



Symmetric vs asymmetric quantization - asymmetric is common for activations to capture full range

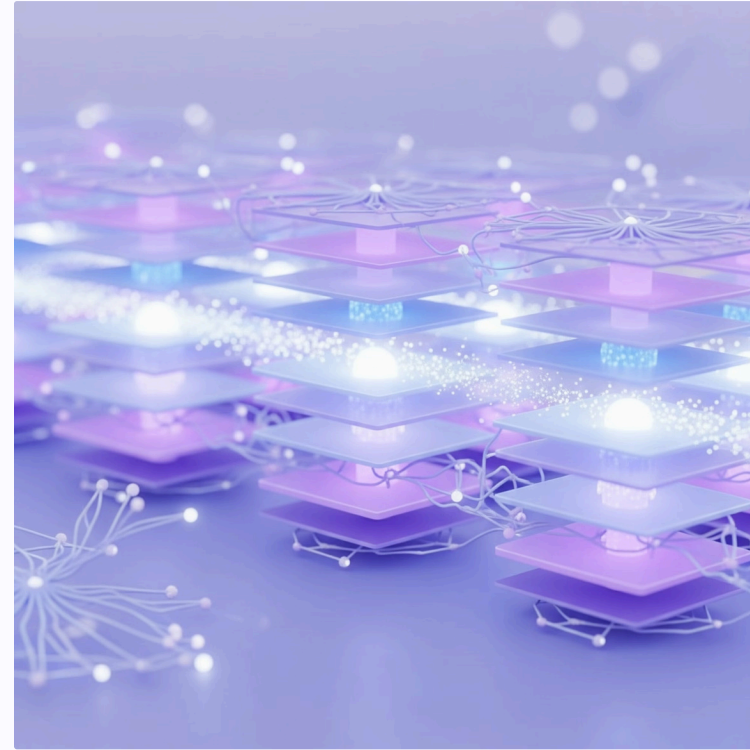
Quantization Approaches

PTQ (Post-Training Quantization)



- No retraining required; quick to implement
- Requires calibration data (representative sample)
- Best for well-behaved networks (ConvNets, some Transformers)
- Minimal engineering effort; good first approach

QAT (Quantization-Aware Training)



- Simulates quantization effects during training
- Higher accuracy, especially for INT8/INT4 or sensitive models
- Requires more engineering effort and training time
- Essential for preserving accuracy in aggressive quantization

Hardware Landscape for Quantized Models



ARM CPUs (NEON)

INT8 fast paths available in TFLite and ONNX Runtime

Common in smartphones and mainstream edge devices



NPUs/Edge TPUs

INT8 native acceleration (Coral, mobile SoCs)

Specifically designed for low-precision inference



NVIDIA Jetson

TensorRT INT8 with calibration/QAT support

Mixed precision capabilities for optimal performance



Apple/Android

Core ML / NNAPI backends prefer low-precision

Optimized for mobile hardware acceleration



PTQ Quick Start (PyTorch → ONNX Runtime)

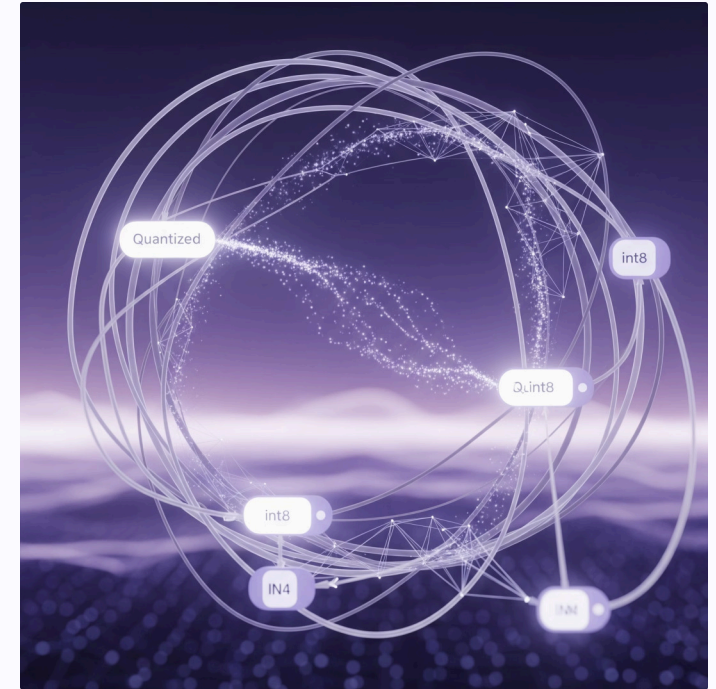
```
# Collect a small calibration set (100-1000 samples)
calib = [prepare_input(x) for x in calib_loader]

# Export to ONNX
torch.onnx.export(model, dummy, "model_fp32.onnx",
                  opset_version=17)

# ONNX Runtime static PTQ (simplified sketch)
from onnxruntime.quantization import quantize_static, \
    CalibrationDataReader, QuantType

class MyCalib(CalibrationDataReader):
    def get_next(self):
        # yield dicts: {input_name: np_array}
        for x in calib:
            yield { "input": x }

quantize_static(
    "model_fp32.onnx",
    "model_int8.onnx",
    MyCalib(),
    weight_type=QuantType.QInt8,
    optimize_model=True
)
```



Result: model_int8.onnx with INT8 weights/activations, typically delivering **2-4× faster inference** on edge devices.

QAT Quick Start (PyTorch)

```
import torch, torch.ao.quantization as tq

model.train()
model.fuse_model() # if supported (Conv+BN+ReLU)

qconfig = tq.get_default_qat_qconfig("fbgemm")
model.qconfig = qconfig
tq.prepare_qat(model, inplace=True)

for step, (x, y) in enumerate(train_loader):
    loss = train_step(model, x, y) # standard training

    if step % 1000 == 0:
        validate(model)

model.eval()
tq.convert(model, inplace=True) # produces INT8 modules
```



Tip: Run a brief fine-tune (e.g., 5–20% of original training epochs) for best accuracy-efficiency tradeoff.

TFLite PTQ (TensorFlow/Keras)

```
import tensorflow as tf

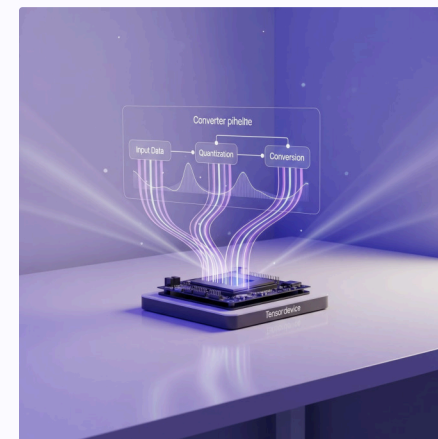
converter = tf.lite.TFLiteConverter.from_saved_model(
    "saved_model_dir")

converter.optimizations = [tf.lite.Optimize.DEFAULT]

# Representative dataset for calibration
def rep_data():
    for x in calib_ds.take(200):
        yield [x.astype('float32')]

converter.representative_dataset = rep_data

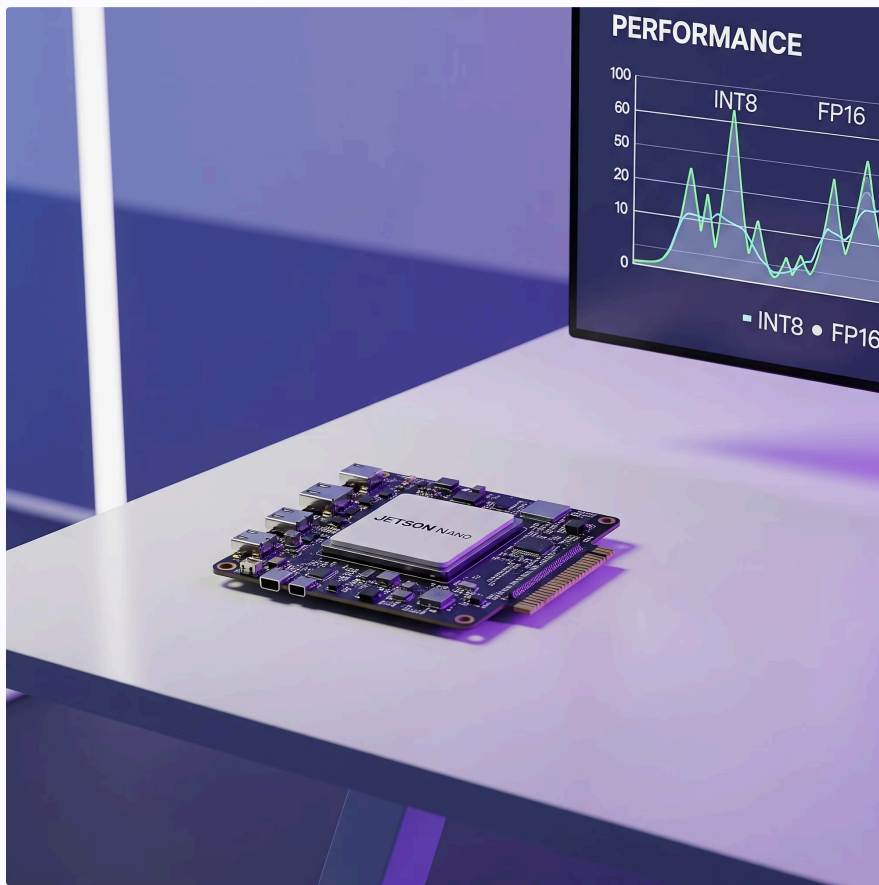
tflite_model = converter.convert()
open("model_int8.tflite", "wb").write(tflite_model)
```



- ❑ **Optional full-int8:** Add these settings to constrain input/output types for Edge TPU/NPUs compatibility:

```
converter.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
```

TensorRT INT8 (Jetson)



Export and Build

Export ONNX → build **INT8 engine** with calibration dataset

Mixed Precision

Sensitive layers can remain **FP16** for accuracy preservation

Performance Gain

Expect **1.5-3×** speedups vs FP16 on supported operations

Validation Strategy

Monitor accuracy drop; adjust **per-channel** and **calibration** strategy as needed

Accuracy Guardrails

Representative Calibration

Use data that accurately covers your real distribution. Poor calibration data = poor quantization.

Per-Channel Quantization

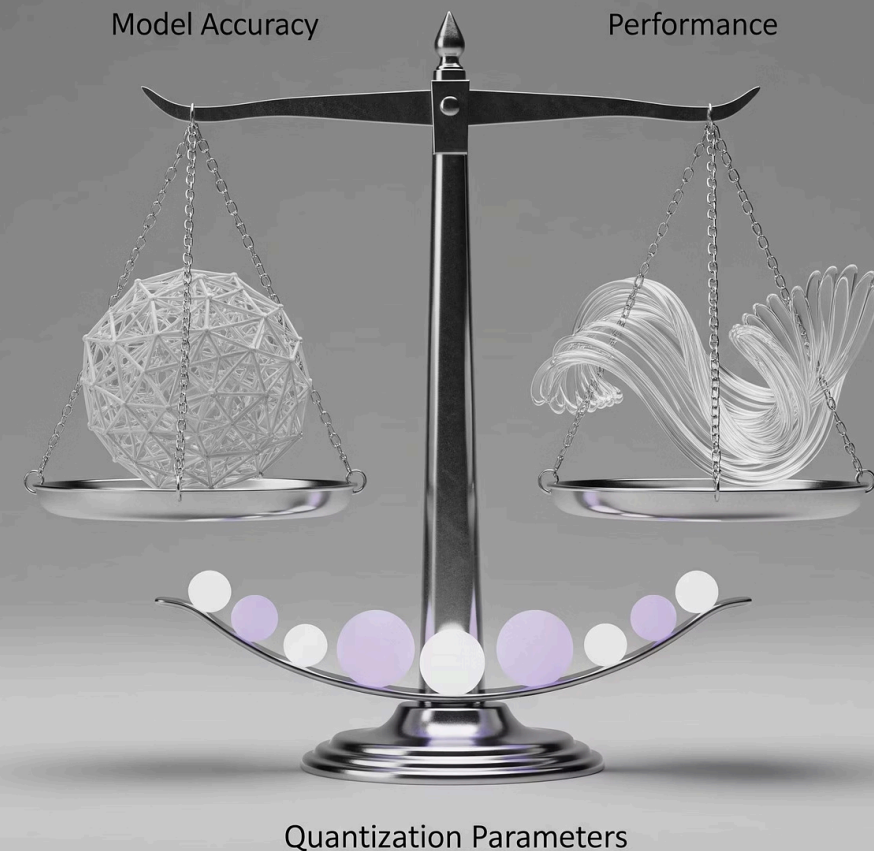
Enable for convolutional weights to preserve channel-specific dynamic ranges and prevent accuracy degradation.

Skip Sensitive Operations

Don't quantize overly sensitive ops like layer normalization and attention softmax which rely on precise values.

Special Transformer Techniques

For Transformer models: QAT or [smooth quant](#)/log-scaled activations help tame outlier values.



Validation Checklist

1

Accuracy Metrics

Measure **top-1 / F1 / WER** delta vs FP32 (target drop $\leq 1\%$)

2

Performance Metrics

Compare **p50/p95 latency** and **throughput** on target device

3

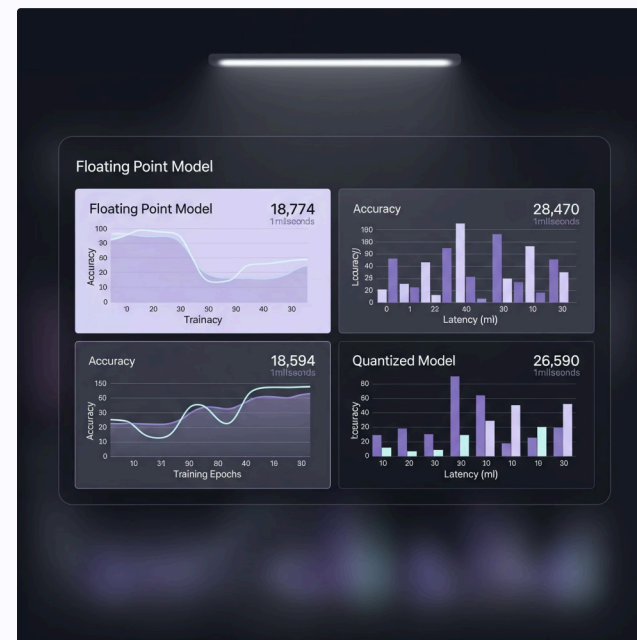
Resource Usage

Track **power draw** and **thermal headroom** during extended inference

4

Production Testing

A/B test in production with guardrails & automated rollback capability



⚠ Remember: Device-specific validation is critical! Performance characteristics can vary dramatically between hardware targets.



Common Pitfalls & Fixes

Big Accuracy Drop

Use QAT instead of PTQ, improve calibration dataset diversity, exclude sensitive layers from quantization

Layer Mismatch After Export

Align ONNX opset versions & fusion patterns, update runtime libraries to latest versions

Edge Device OOM

Quantize **activations** too (not just weights), reduce batch size, use INT8 I/O types

Jittery Latency

Pin CPU threads, use static input shapes, implement proper warmup before timing

When to Go Beyond INT8



FP16/...

Quick Win

Minimal accuracy loss,
great on GPUs and
modern NPUs

Nearly universal support
across frameworks

INT4

Aggressive Compression

Consider QAT + per-
channel + activation
clipping

Best for research and
specialized NPU hardware

W8A16

Weight-only Quantization

For LLMs: KV cache &
matmul-int operations

Pair with techniques like
AWQ/GPTQ for optimal
results

Select precision based on hardware capabilities and accuracy requirements. More aggressive quantization requires more careful validation.

Key Takeaways

Highest Impact Optimization

Quantization is the **#1 lever** for improving edge latency, power consumption, and memory footprint

Approach Selection

PTQ is fast to implement and evaluate; **QAT** restores accuracy on challenging models

Runtime Integration

Pair with **hardware-specific runtimes** (TFLite, TensorRT, NNAPI, Core ML) for maximum benefit

Continuous Validation

Validate thoroughly; automate performance and accuracy measurement in CI for every model release

